CHAKOTEAM! BOOKHAND

General
Template 2

Grafos
BellmanFord 2
BinaryLifting 2
Djikstra 2
Floyd-Warshall 2
Kruskal 2
Lca 3

Flujo Y Conectividad
2-sat 3
Camino Dirigido Euler 4
Camino No Dirigido Euler 4
Dinic 5
Tarjan 5
TarjanSCC 5

Estructuras De Datos
Fenwick 6
Fenwick 2D 6
Fenwick Walk 6
Line Container 6
Mo 6
Segment Tree 7
Segment Tree Lazy 7
Segment Tree Walk 8
Sparse Table 8
Sqrt Decomposition 8
Union Find 8

Matematicas
DiscreteRoot 9
GeometryStructures 9
InclusionExclusion 10
SimpsonRule 10
TonelliShanks 10
Ztransform 11
Bin Pow 11
CatalanNumbers 11
Convex Hull 11
Crt 12
Discrete Log 12
Erathostenes-sieve 12
Factorization Pollard Rho 13
Fft 13
Gaussian Elimination 14
Gcd 14
GcdExtended 14
Miller Rabin 14
Mobius 14
Ntt 15
Primitive Root 15
Template 15
Tot Euler 16

Strings
Aho-corasick 16
Kmp 17
RollingHashing 17
Trie 17
Zfunction 17

Programacion Dinamica
Knappsack 17
Lis 17
Lis Length K 17

Miscelaneo
Busqueda Binaria 18
Random 18
Time 18

General

Template

```cpp
#pragma GCC optimize("O3", "unroll-loops")
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define endl '\n'
#define all(v) (v).begin(), (v).end()

void solve(){

}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    ll t = 1; cin >> t;
    while(t--) solve();
}
```

Grafos

BellmanFord

```cpp
struct BellmanFord {
  struct Edge {
    int from, to, weight;
  };
  int n, last_updated = -1;
  const ll INF = 1e18;
  vector<int> p, dist;
  BellmanFord(vector<Edge>& G, int s, int nn) {
    n = nn;
    dist.assign(n + 2, INF);
    p.assign(n + 2, -1);
    dist[s] = 0;
    for (int i = 1; i <= n; i++) {
      last_updated = -1;
      for (Edge& e : G)
        if (dist[e.from] + e.weight < dist[e.to]) {
          dist[e.to] = dist[e.from] + e.weight;
          p[e.to] = e.from;
          last_updated = e.to;
        }
    }
  }
  bool getCycle(vector<int>& cycle) {
    if (last_updated == -1) return false;
    for (int i = 0; i < n - 1; i++) last_updated = p[last_updated];
    for (int x = last_updated;; x = p[x]) {
      cycle.push_back(x);
      if (x == last_updated and cycle.size() > 1) break;
    }
    reverse(cycle.begin(), cycle.end());
    return true;
  }
};
```

BinaryLifting

```cpp
void BinaryLifting(ll node) {
  stack<pair<ll, ll>> st;
  st.push({node, -1});
  while (st.size()) {
    auto [n, p] = st.top();
    st.pop();
    for (auto v : tree[n]) {
      if (p == v) continue;
      st.push({v, n});
      bl[v][0] = n;
      for (ll j = 1; j < 33; j++) {
        if (bl[v][j - 1] != -1) {
          bl[v][j] = bl[bl[v][j - 1]][j - 1];
        } else {
          bl[v][j] = -1;
        }
      }
    }
  }
}
```

Djikstra

```cpp
vector<ll> dist;
vector < vector<pair<ll, ll>> g;

void djikstra() {
  priority_queue<pair<ll, ll>, vector<pair<ll, ll>>, greater<pair<ll, ll>>> pq;
  pq.push({0, 0});  // nodo_inicio, distancia
  while (pq.size()) {
    auto [c, node] = pq.top();
    pq.pop();
    if (dist[node] <= c) continue;
    dist[node] = c;
    for (auto [c_, v_] : g[node]) {
      if (dist[v_] > c + c_) pq.push({c + c_, v_});
    }
  }
}
```

Floyd-Warshall

```cpp
for (int i = 1; i <= n; i++) {
  for (int j = 1; j <= n; j++) {
    if (i == j)
      distance[i][j] = 0;
    else if (adj[i][j])
      distance[i][j] = adj[i][j];
    else
      distance[i][j] = INF;
  }
}
for (int k = 1; k <= n; k++) {
  for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
      distance[i][j] = min(distance[i][j], distance[i][k] + distance[k][i]);
    }
  }
}
```

Kruskal

```cpp
struct Edge {
  int a;
  int b;
  ll w;
  Edge(int a_, int b_, ll w_) : a(a_), b(b_), w(w_) {}
};
bool c_edge(Edge& a, Edge& b) { return a.w < b.w; }
ll Kruskal() {
  int n = G.size();
  union_find sets(n);
  vector<Edge> edges;
  for (int i = 0; i < n; i++) {
    for (auto eg : G[i]) {
      // node i to node eg.first with cost eg.second
      edges.emplace_back(i, eg.first, eg.second);
    }
  }
  sort(edges.begin(), edges.end(), c_edge);
  ll min_cost = 0;
  for (Edge e : edges) {
    if (sets.sameSet(e.a, e.b) != true) {
      tree.emplace_back(e.a, e.b, e.w);
      min_cost += e.w;
      sets.unionSet(e.a, e.b);
    }
  }
  return min_cost;
}
```

Lca

```cpp
struct LCA {
  vector<vector<ll>> T, parent;
  vector<ll> depth, vis, comp;
  ll LOGN, V;
  LCA(vector<vector<ll>>& T) : T(T) {
    V = (ll)T.size();
    LOGN = 1;
    while ((1LL << LOGN) <= max(1LL, V)) LOGN++;
    parent.assign(V, vector<ll>(LOGN, 0));
    depth.assign(V, 0);
    vis.assign(V, 0);
    comp.assign(V, -1);
    ll id = 0;
    for (ll u = 0; u < V; u++) {
      if (!vis[u]) {
        dfs(u, u, id++);
      }
    }
  }

  void dfs(ll u, ll p, ll id) {
    vis[u] = 1;
    comp[u] = id;
    parent[u][0] = p;
    for (ll j = 1; j < LOGN; j++)
      parent[u][j] = parent[parent[u][j - 1]][j - 1];

    for (ll v : T[u]) {
      if (!vis[v]) {
        depth[v] = depth[u] + 1;
        dfs(v, u, id);
      }
    }
  }
  ll query(ll u, ll v) {
    if (comp[u] != comp[v]) return -1;
    if (depth[u] < depth[v]) swap(u, v);
    ll k = depth[u] - depth[v];
    for (ll j = LOGN - 1; j >= 0; j--) {
      if ((k >> j) & 1) u = parent[u][j];
    }
    if (u == v) return u;
    for (ll j = LOGN - 1; j >= 0; j--) {
      if (parent[u][j] != parent[v][j]) {
        u = parent[u][j];
        v = parent[v][j];
      }
    }
    return parent[u][0];
  }
};
```

Flujo Y Conectividad

2-sat

```cpp
struct TwoSat {
  int N;
  vector<vector<int>> gr;
  vector<int> values;
  TwoSat(int n = 0) : N(n), gr(2 * n) {}
  int addVar() {
    gr.emplace_back();
    gr.emplace_back();
    return N++;
  }
  void either(int f, int j) {
    f = max(2 * f, -1 - 2 * f);
    j = max(2 * j, -1 - 2 * j);
    gr[f].push_back(j ^ 1);
    gr[j].push_back(f ^ 1);
  }
  void setValue(int x) { either(x, x); }
  void atMostOne(const vector<int>& li) {
    if ((int)li.size() <= 1) return;
    int cur = ~li[0];
    for (int i = 2; i < (int)li.size(); i++) {
      int next = addVar();
      either(cur, ~li[i]);
      either(cur, next);
      either(~li[i], next);
      cur = ~next;
    }
    either(cur, ~li[1]);
  }
  vector<int> val, comp, z;
  int Time = 0;
  int dfs(int i) {
    int low = val[i] = ++Time, x;
    z.push_back(i);
    for (int e : gr[i])
      if (!comp[e]) low = min(low, val[e] ? val[e] : dfs(e)
);
    if (low == val[i]) do {
        x = z.back();
        z.pop_back();
        comp[x] = low;
        if (values[x >> 1] == -1) values[x >> 1] = x & 1;
      } while (x != i);
    return val[i] = low;
  }
  bool solve() {
    values.assign(N, -1);
    val.assign(2 * N, 0);
    comp = val;
    for (int i = 0; i < 2 * N; i++)
      if (!comp[i]) dfs(i);
    for (int i = 0; i < N; i++)
      if (comp[2 * i] == comp[2 * i + 1]) return false;
    return true;
  }
};
```

Camino Dirigido Euler

```cpp
struct directed_euler_trail {
  ll n, m = 0;
  vector<vector<ll>> adj;
  vector<pair<ll, ll>> E;
  vector<ll> indeg, outdeg, vertices, edges;
  directed_euler_trail(ll n) : n(n), adj(n), indeg(n), outdeg(n) {}
  void add_edge(ll u, ll v) {
    adj[u].push_back(m);
    E.push_back({u, v});
    outdeg[u]++, indeg[v]++, m++;
  }
  ll find_start() {
    ll s = -1, cp = 0, cm = 0;
    for (ll v = 0; v < n; v++) {
      ll d = outdeg[v] - indeg[v];
      if (d == 1)
        cp++, s = v;
      else if (d == -1)
        cm++;
      else if (d != 0)
        return -2;
      if (outdeg[v] > 0 && s < 0) s = v;
    }
    if (cp == 0 && cm == 0) return s;
    return (cp == 1 && cm == 1) ? s : -2;
  }
  bool solve() {
    if (m == 0) {
      vertices = {0};
      return true;
    }
    ll s = find_start();
    if (s == -2) return false;
    vector<ll> ep(n);
    edges.resize(m);
    ll pp = 0, pq = m, v = s;
    while (pp < pq) {
      if (ep[v] >= (ll)adj[v].size()) {
        if (pp == 0) return false;
        edges[--pq] = edges[--pp];
        v = E[edges[pq]].first;
      } else {
        ll e = adj[v][ep[v]++];
        edges[pp++] = e;
        v = E[e].second;
      }
    }
    vertices = {s};
    for (ll e : edges) vertices.push_back(E[e].second);
    return true;
  }
};
```

Camino No Dirigido Euler

```cpp
struct undirected_euler_trail {
  ll n, m = 0;
  vector<vector<ll>> adj;
  vector<pair<ll, ll>> E;
  vector<ll> deg, vertices, edges;

  undirected_euler_trail(ll n) : n(n), adj(n), deg(n) {}

  void add_edge(ll u, ll v) {
    adj[u].push_back(m);
    if (u != v) adj[v].push_back(m);
    E.push_back({u, v});
    deg[u]++, deg[v]++, m++;
  }

  ll find_start() {
    ll s = -1, codd = 0;
    for (ll v = 0; v < n; v++) {
      if (deg[v] & 1)
        codd++, s = v;
      else if (deg[v] > 0 && s < 0)
        s = v;
    }
    return (codd == 0 || codd == 2) ? s : -2;
  }

  bool solve() {
    if (m == 0) {
      vertices = {0};
      return true;
    }

    ll s = find_start();
    if (s == -2) return false;

    vector<ll> ep(n);
    edges.resize(m);
    vector<bool> alive(m, true);
    ll pp = 0, pq = m, v = s;

    while (pp < pq) {
      if (ep[v] >= adj[v].size()) {
        if (pp == 0) return false;
        edges[--pq] = edges[--pp];
        v ^= E[edges[pq]].first ^ E[edges[pq]].second;
      } else {
        ll e = adj[v][ep[v]++];
        if (!alive[e]) continue;
        alive[e] = false;
        edges[pp++] = e;
        v ^= E[e].first ^ E[e].second;
      }
    }
    vertices = {s};
    for (ll e : edges) {
      s ^= E[e].first ^ E[e].second;
      vertices.push_back(s);
    }
    return true;
  }
};
```

Dinic

```cpp
template <class T>
struct Dinic {
  struct Edge {
    int to, rev;
    T f, c;
    bool r;
  };
  int n, t_;
  vector<vector<Edge>> G;
  vector<pair<ll, ll>> min_cut;
  vector<int> D, q, W;
  bool bfs(int s, int t) {
    W.assign(n, 0);
    D.assign(n, -1);
    D[s] = 0;
    int f = 0, l = 0;
    q[l++] = s;
    while (f < l) {
      int u = q[f++];
      for (const Edge& e : G[u])
        if (D[e.to] == -1 && e.f < e.c) D[e.to] = D[u] + 1, q[l++] = e.to;
    }
    return D[t] != -1;
  }
  int dfs(int u, T f) {
    if (u == t_) return f;
    for (int& i = W[u]; i < (int)G[u].size(); ++i) {
      Edge& e = G[u][i];
      int v = e.to;
      if (e.c <= e.f || D[v] != D[u] + 1) continue;
      T df = dfs(v, min(f, e.c - e.f));
      if (df > 0) {
        e.f += df, G[v][e.rev].f -= df;
        return df;
      }
    }
    return 0;
  }
  Dinic(int N) : n(N), G(N), D(N), q(N) {}
  void add_edge(int u, int v, T cap) {
    G[u].push_back({v, (int)G[v].size(), 0, cap, 0});
    G[v].push_back({u, (int)G[u].size() - 1, 0, 0, 1});  // Use cap instead of 0 if bidirectional
  }
  T max_flow(int s, int t) {
    t_ = t;
    T ans = 0;
    while (bfs(s, t))
      while (T dl = dfs(s, numeric_limits<T>::max())) ans += dl;
    return ans;
  }
  void solve_min_cut(vector<pair<ll, ll>>& edges) {
    for (auto [u, v] : edges) {
      if ((D[u] != -1) != (D[v] != -1)) {
        min_cut.emplace_back(u, v);
      }
    }
  }
};
```

Tarjan

```cpp
struct tarjan {
  struct edge {
    int u, v, comp, bridge;
  };
  int n, nbc, T;
  vector<vector<int>> adj;
  vector<edge> e;
  vector<int> disc, low, art, st;
  tarjan(int n) : n(n), nbc(0), T(0), adj(n), disc(n, -1), low(n), art(n, 0) {}
  int add_edge(int u, int v) {
    int i = e.size();
    adj[u].push_back(i);
    adj[v].push_back(i);
    e.push_back({u, v, -1, false});
    return i;
  }
  void dfs(int u, int pe) {
    low[u] = disc[u] = T++;
    for (int i : adj[u]) {
      if (i == pe) continue;
      int v = e[i].u ^ e[i].v ^ u;
      if (disc[v] < 0) {
        st.push_back(i);
        dfs(v, i);
        if (low[v] > disc[u]) e[i].bridge = true;
        if (low[v] >= disc[u]) {
          art[u]++;
          while (st.back() != i) {
            e[st.back()].comp = nbc;
            st.pop_back();
          }
          e[st.back()].comp = nbc;
          st.pop_back();
          nbc++;
        }
        low[u] = min(low[u], low[v]);
      } else if (disc[v] < disc[u]) {
        st.push_back(i);
        low[u] = min(low[u], disc[v]);
      }
    }
  }
  void solve() {
    for (int i = 0; i < n; i++)
      if (disc[i] < 0) dfs(i, -1), art[i]--;
  }
  bool is_bridge(int i) { return e[i].bridge; }
  bool is_art(int v) { return art[v] > 0; }
  int component(int i) { return e[i].comp; }
};
```

TarjanSCC

```cpp
struct tarjan {
  int n, timer;
  vector<bool> vis, ist;
  vector<int> tin, low, st, cmp;
  vector<vector<int>> scc;
  void dfs(int u, vector<vector<int>>& adj) {
    vis[u] = ist[u] = 1;
    st.push_back(u);
    tin[u] = low[u] = timer++;
    for (auto v : adj[u]) {
      if (!vis[v]) {
        dfs(v, adj);
        low[u] = min(low[u], low[v]);
      } else if (ist[v]) {
        low[u] = min(low[u], tin[v]);
      }
    }
    if (low[u] == tin[u]) {
      scc.emplace_back();
      int cur;
      do {
        cur = st.back();
        st.pop_back();
        ist[cur] = 0;
        scc.back().push_back(cur);
        cmp[cur] = (int)scc.size() - 1;
      } while (cur != u);
    }
  }
  tarjan(vector<vector<int>>& adj) {
    n = adj.size();
    tin.resize(n);
    timer = 0;
    cmp.resize(n);
    low.resize(n);
    vis.resize(n, 0);
    ist.resize(n, 0);
    for (int i = 0; i < n; i++)
      if (!vis[i]) dfs(i, adj);
  }
};
```

Fenwick

```cpp
struct fenwick_tree {
  vector<ll> bit;
  int n;
  fenwick_tree(int n) : n(n) { bit.assign(n, 0); }
  fenwick_tree(vector<ll>& a) : fenwick_tree(a.size()) {
    for (size_t i = 0; i < a.size(); i++) {
      bit[i] += a[i];
      if ((i | (i + 1)) < a.size()) bit[i | (i + 1)] += bit[i];
    }
  }
  ll sum(int r) {
    ll ret = 0;
    for (; r >= 0; r = (r & (r + 1)) - 1) ret += bit[r];
    return ret;
  }
  ll sum(int l, int r) { return sum(r) - sum(l - 1); }
  void add(int idx, ll delta) {
    for (; idx < n; idx = idx | (idx + 1)) bit[idx] += delta;
  }
};
```

Fenwick 2D

```cpp
struct fenwick_tree_2d {
  int N, M;
  vector<vector<ll>> BIT;
  fenwick_tree_2d(int N, int M) : N(N), M(M) {
    BIT.assign(N + 1, vector<ll>(M + 1, 0));
  }
  void update(int x, int y, ll v) {
    for (int i = x; i <= N; i += (i & -i))
      for (int j = y; j <= M; j += (j & -j)) BIT[i][j] += v;
  }
  ll sum(int x, int y) {
    ll s = 0;
    for (int i = x; i > 0; i -= (i & -i))
      for (int j = y; j > 0; j -= (j & -j)) s += BIT[i][j];
    return s;
  }
  ll query(int x1, int y1, int x2, int y2) {
    return sum(x2, y2) - sum(x2, y1 - 1) - sum(x1 - 1, y2) +
           sum(x1 - 1, y1 - 1);
  }
};
```

Fenwick Walk

```cpp
struct fenwick_tree_with_walk : fenwick_tree {
  ll walk(ll k) {
    if (k <= 0 || k > sum(n - 1)) return n;
    ll i = -1, s = 1;
    while ((s << 1) < n) s <<= 1;
    while (s > 0) {
      ll m = i + s;
      if (m < n && bit[m] < k) {
        i = m;
        k -= bit[m];
      }
      s >>= 1;
    }
    return i + 1;
  }
};
```

Line Container

```cpp
const ll INF = 1e15;
struct Line {
  mutable ll a, b, c;
  bool operator<(Line r) const {
    return a < r.a;
  }
  bool operator<(ll x) const {
    return c < x;
  }
};
// dynamically insert `a*x + b` lines and query for maximum
// at any x all operations have complexity O(log N)
struct LineContainer: multiset < Line, less < >> {
  ll div(ll a, ll b) {
    return a / b - ((a ^ b) < 0 && a % b);
  }
  bool isect(iterator x, iterator y) {
    if (y == end()) return x -> c = INF, 0;
    if (x -> a == y -> a) x -> c = x -> b > y -> b ? INF : -INF;
    else x -> c = div(y -> b - x -> b, x -> a - y -> a);
    return x -> c >= y -> c;
  }

  void add(ll a, ll b) {
    // a *= -1, b *= -1 // for min
    auto z = insert({
      a,
      b,
      0
    }), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
    while ((y = x) != begin() && (--x) -> c >= y -> c) isect(x, erase(y));
  }

  ll query(ll x) {
    if (empty()) return -INF; // INF for min
    auto l = * lower_bound(x);
    return l.a * x + l.b;
    // return -l.a * x - l.b; // for min
  }
};
```

Mo

```cpp
struct query {
  ll l, r, i;
};
template<class T, typename U, class T2>
struct mo_algorithm {
  vector<U> ans;
  template<typename... Args>
  mo_algorithm(vector<T> &v, vector<query> &queries, Args... args) {
    T2 ds(args...);
    ll block_sz = sqrtl(v.size());
    ans.assign(queries.size(), -1);
    sort(queries.begin(), queries.end(), [&](auto &a, auto &b) {
      return a.l/block_sz != b.l/block_sz ? a.l/block_sz < b.l/block_sz : a.r < b.r;
    });
    ll l = 0, r = -1;
    for (query q : queries) {
      while (l > q.l) ds.add(v[--l]);
      while (r < q.r) ds.add(v[++r]);
      while (l < q.l) ds.remove(v[l++]);
      while (r > q.r) ds.remove(v[r--]);
      ans[q.i] = ds.answer(q);
    }
  }
};
```

Segment Tree

```cpp
template<class T, T m_(T, T)> struct segment_tree{
  int n; vector<T> ST;
  segment_tree(){}
  segment_tree(vector<T> &a){
    n = a.size(); ST.resize(n << 1);
    for (int i=n;i<(n<<1);i++)ST[i]=a[i-n];
    for (int i=n-1;i>0;i--)ST[i]=m_(ST[i<<1],ST[i<<1|1]);
  }
  void update(int pos, T val){ // replace with val
    ST[pos += n] = val;
    for (pos >>= 1; pos > 0; pos >>= 1)
      ST[pos] = m_(ST[pos<<1], ST[pos<<1|1]);
  }
  T query(int l, int r){ // [l, r]
    T ansL, ansR; bool hasL = 0, hasR = 0;
    for (l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
      if (l & 1)
        ansL=(hasL?m_(ansL,ST[l++]):ST[l++]),hasL=1;
      if (r & 1)
        ansR=(hasR?m_(ST[--r],ansR):ST[--r]),hasR=1;
    }
    if (!hasL) return ansR; if (!hasR) return ansL;
    return m_(ansL, ansR);
  }
};
```

Segment Tree Lazy

```cpp
template<
  class T1, // answer value stored on nodes
  class T2, // lazy update value stored on nodes
  T1 merge(T1, T1),
  void pushUpd(T2& /*parent*/, T2& /*child*/, int, int, int, int), // push update value from a node to another. parent -> child
  void applyUpd(T2&, T1&, int, int)           // apply the update value of a node to its answer value. upd -> ans
>
struct segment_tree_lazy{
  vector<T1> ST; vector<T2> lazy; vector<bool> upd;
  int n;
  void build(int i, int l, int r, vector<T1>&values){
    if (l == r){
      ST[i] = values[l];
      return;
    }
    build(i << 1, l, (l + r) >> 1, values);
    build(i << 1 | 1, (l + r) / 2 + 1, r, values);
    ST[i] = merge(ST[i << 1], ST[i << 1 | 1]);
  }
  segment_tree_lazy() {}
  segment_tree_lazy(vector<T1>&values){
    n = values.size(); ST.resize(n << 2 | 3);
    lazy.resize(n << 2 | 3); upd.resize(n << 2 | 3, false);
    build(1, 0, n - 1, values);
  }
  void push(int i, int l, int r){
    if (upd[i]){
      applyUpd(lazy[i], ST[i], l, r);
      if (l != r){
        pushUpd(lazy[i], lazy[i << 1], l, r, l, (l + r) / 2);
        pushUpd(lazy[i], lazy[i << 1 | 1], l, r, (l + r) / 2 + 1, r);
        upd[i << 1] = 1;
        upd[i << 1 | 1] = 1;
      }
      upd[i] = false;
      lazy[i] = T2();
    }
  }
  void update(int i, int l, int r, int a, int b, T2 &u){
    if (l >= a and r <= b){
      pushUpd(u, lazy[i], a, b, l, r);
      upd[i] = true;
    }
    push(i, l, r);
    if (l > b or r < a) return;
    if (l >= a and r <= b) return;
    update(i << 1, l, (l + r) >> 1, a, b, u);
    update(i << 1 | 1, (l + r) / 2 + 1, r, a, b, u);
    ST[i] = merge(ST[i << 1], ST[i << 1 | 1]);
  }
  void update(int a, int b, T2 u){
    if (a > b){
      update(0, b, u);
      update(a, n - 1, u);
      return ;
    }
    update(1, 0, n - 1, a, b, u);
  }
  T1 query(int i, int l, int r, int a, int b){
    push(i, l, r);
    if (a <= l and r <= b)
      return ST[i];
    int mid = (l + r) >> 1;
    if (mid < a)
      return query(i << 1 | 1, mid + 1, r, a, b);
    if (mid >= b)
      return query(i << 1, l, mid, a, b);
    return merge(query(i << 1, l, mid, a, b), query(i << 1 | 1, mid + 1, r, a, b));
  }
  T1 query(int a, int b){
    if (a > b) return merge(query(a, n - 1), query(0, b));
    return query(1, 0, n - 1, a, b);
  }
};
```

Segment Tree Walk

```cpp
template<class T, T m_(T, T)> struct segment_tree_with_walk
 : segment_tree<T,m_> {
  int walk(int l, T k) {
    vector<int> L, R;
    int r = this->n - 1;
    for (l += this->n, r += this->n + 1; l < r; l >>= 1, r
>>= 1) {
      if (l & 1) L.push_back(l++);
      if (r & 1) R.push_back(--r);
    }
    reverse(R.begin(), R.end());
    L.insert(L.end(), R.begin(), R.end());
    for (int v : L) {
      if (this->ST[v] >= k) {
        while (v < this->n) v = (this->ST[v << 1] >= k) ? (
v << 1) : (v << 1 | 1);
        return v - this->n;
      }
    }
    return -1;
  }
};
```

Sparse Table

```cpp
template <typename T, T m_(T, T)>
struct sparse_table {
  int n;
  vector<vector<T>> table;
  sparse_table() {}
  sparse_table(vector<T> &arr) {
    n = arr.size();
    int k = log2_floor(n) + 1;
    table.assign(n, vector<T>(k));
    for (int i = 0; i < n; i++)
      table[i][0] = arr[i];
    for (int j = 1; j < k; j++)
      for (int i = 0; i + (1 << j) <= n; i++)
        table[i][j] = m_(table[i][j-1], table[i+(1<<(j-1))]
[j-1]);
  }
  T query(int l, int r) {
    int k = log2_floor(r - l + 1);
    return m_(table[l][k], table[r-(1 << k)+1][k]);
  }
  int log2_floor(int n) { return n ? __builtin_clzll(1) - _
_builtin_clzll(n) : -1; }
};
```

Sqrt Decomposition

```cpp
template <class T, T merge(T,T)>
struct sqrt_decomp {
  int n,sq;
  vector<T> st,bl;
  sqrt_decomp(vector<T> & a){
    n = a.size(); st.resize(n);
    sq = sqrtl(n); bl.resize((n+sq-1)/sq);
    for (int i = 0; i < n; i++){
      st[i] = a[i];
      bl[i/sq] = ((i%sq == 0)?st[i]:merge(bl[i/sq],st[i]));
    }
  }
  void update(int pos, T val) {
    st[pos] = val;
    int block = pos/sq, lb = block*sq, rb = min(block*sq+sq
-1,n-1);
    T vl;
    for (int i = lb; i <= rb; i++)
      vl = ((i == lb)?st[i]:merge(vl,st[i]));
    bl[block] = vl;
  }
  T query(int l, int r){
    T res; bool hasR = 0;
    for (int i = l/sq; i <= r/sq; i++){
      int lb = i*sq, rb = i*sq+sq-1;
      if (l <= lb && rb <= r){
        res = (hasR?merge(res,bl[i]):bl[i]), hasR = 1;
        continue;
      }
      for (int j = max(l,lb); j <= min(rb,r); j++)
        res = (hasR?merge(res,st[j]):st[j]), hasR = 1;
    }
    return res;
  }
};
```

Union Find

```cpp
struct union_find {
  vector<int> e;
  union_find(int n) { e.assign(n, -1); }
  int findSet(int x) { return (e[x] < 0 ? x : e[x] = findSe
t(e[x])); }
  bool sameSet(int x, int y) { return findSet(x) == findSet
(y); }
  int size(int x) { return -e[findSet(x)]; }
  bool unionSet(int x, int y) {
    x = findSet(x), y = findSet(y);
    if (x == y) return 0;
    if (e[x] > e[y]) swap(x, y);
    e[x] += e[y], e[y] = x;
    return 1;
  }
};
```

Matematicas

DiscreteRoot

```cpp
#include<bits/stdc++.h>

using namespace std;

#define ll long long
#define ull unsigned ll
#define for_(i, n) for(ll i = 0; i < n; ++i)
#define INF LLONG_MAX
#define NINF LLONG_MIN

const ll mod = 1e9 + 7;

//TESTED

ll gcd(ll a, ll b) {
    if(b==0) return a;
    return gcd(b, a%b);
}

ll binpow(ll a, ll b, ll m){
    ll ans = 1;
    while(b > 0){
        if(b&1) ans = (ans*a)%m;
        a=(a*a)%m;
        b>>=1;
    }
    return ans;
}

vector<ll> tot_euler(ll n){
    vector<ll> tot(n+1, 0);
    for_(i, n+1) tot[i]=i;
    for(ll i = 2; i <= n; ++i){
        if(tot[i] == i){
            for(ll j = i; j <= n; j+=i) tot[j] -= tot[j]/i;
        }
    }
    return tot;
}

ll generator_(ll n) {
    vector<int> facts;
    ll phi_n = tot_euler(n)[n], m = phi_n;
    for(ll i=2; i*i<=m; ++i){
        if(m % i == 0) {
            facts.push_back(i);
            while (m % i == 0) m/= i;
        }
    }
    if(m > 1) facts.push_back(m);
    for (ll g=2; g<=n; ++g) {
        bool ok = true;
        for (size_t i=0; i<facts.size() && ok; ++i) ok &= binpow(g, phi_n/facts[i], n) != 1;
        if (ok)  return g;
    }
    return -1;
}

vector<ll> DiscreteRoot(ll n,ll k,ll a){
    if(a==0) return {0};
    ll g = generator_(n);

    ll sq = (ll)sqrt(n) + 1;
    vector<pair<ll, ll>> dec(sq);
    for(ll i = 1; i <= sq; ++i)
        dec[i-1] = {binpow(g, i * sq * k % (n - 1), n), i};
    sort(dec.begin(), dec.end());
    ll h0 = -1;
    for_(i, sq) {
        ll my = binpow(g, i * k % (n - 1), n) * a % n;
        auto it = lower_bound(dec.begin(), dec.end(), make_pair(my, 0));
        if (it != dec.end() && it->first == my) {
            h0 = it->second * sq - i;
            break;
        }
    }
    if(h0 == -1) return {-1};

    ll delta = (n-1)/gcd(k, n-1);
    vector<ll> x;
    for (ll h = h0 % delta; h < n-1; h += delta)
        x.push_back(binpow(g, h, n));
    sort(x.begin(), x.end());
    return x;
}
```

GeometryStructures

```cpp
#include<bits/stdc++.h>

using namespace std;
#define ll long long
#define INF LLONG_MAX

struct p2d {
    ll x, y;
    p2d() {}
    p2d(ll x, ll y): x(x), y(y) {}
    p2d& operator+=(const p2d &p) {
        x += p.x;
        y += p.y;
        return *this;
    }
    p2d& operator-=(const p2d &p) {
        x -= p.x;
        y -= p.y;
        return *this;
    }
    p2d& operator*=(ll c) {
        x *= c;
        y *= c;
        return *this;
    }
    p2d& operator/=(ll c) {
        x /= c;
        y /= c;
        return *this;
    }
    p2d operator+(const p2d &p) const {
        return p2d(*this) += p;
    }
    p2d operator-(const p2d &p) const {
        return p2d(*this) -= p;
    }
    p2d operator*(ll c) const {
        return p2d(*this) *= c;
    }
    p2d operator/(ll c) const {
        return p2d(*this) /= c;
    }
    ll dot(p2d a, p2d b){
        return a.x*b.x + a.y*b.y;
    }
    double norm2(p2d a) {
        return sqrt(dot(a, a));
    }
    double norm2sq(p2d a) {
        return dot(a, a);
    }
    p2d proj(p2d a, p2d b) {
        return b*(dot(a, b) / norm2sq(b));
    }
    double angle(p2d a, p2d b) {
        return acos(dot(a, b) / norm2(a) / norm2(b));
    }
    static ll cross(p2d a, p2d b) {
        return a.x * b.y - a.y * b.x;
    }
    p2d intersect_lines(p2d a1, p2d d1, p2d a2, p2d d2){
        ll crs = cross(d1, d2);
        if(crs==0) return p2d(INF, INF);
        return a1 + d1*(cross(a2 - a1, d2) / crs);
    }
};

struct p3d {
    ll x, y, z;
    p3d() {}
    p3d(ll x,ll y,ll z): x(x), y(y), z(z) {}
    p3d& operator+=(const p3d &p) {
        x += p.x;
        y += p.y;
        z += p.z;
        return *this;
    }
    p3d& operator-=(const p3d &p) {
        x -= p.x;
        y -= p.y;
        z -= p.z;
        return *this;
    }
    p3d& operator*=(ll c) {
        x *= c;
        y *= c;
        z *= c;
        return *this;
    }
    p3d& operator/=(ll c) {
        x /= c;
        y /= c;
        z /= c;
        return *this;
    }
    p3d operator+(const p3d &p) const {
        return p3d(*this) += p;
    }
    p3d operator-(const p3d &p) const {
```

InclusionExclusion

```cpp

#include<bits/stdc++.h>

using namespace std;

#define ll long long
#define ull unsigned long long
#define for_(i, n) for(ll i = 0; i < n; ++i)
#define INF LLONG_MAX
#define NINF LLONG_MIN

const ll mod = 1e9 + 7;

//Example cardinality of all subsets of size 1
ll cardinalitySubset(ll subset){
    return 1;
}

//Template
ll InclusionExclusion(){
    //sets
    ll n = 10;//example
    ll subsets = (1<<n);//total subsets
    ll ans = 0;
    //travel on all the subsets, with a particular subset
    for(ll subset = 0; subset < subsets; subset++){
        //if is odd the subset we sum, otherwise substract, recall, Union of sets = sum sets - intersect sets pairs + intersect group of three - ...
        ans += (__builtin_popcount(subset)&1 ? cardinalitySubset(subset) : -cardinalitySubset(subset));
    }
    return ans;
}
```

SimpsonRule

```cpp

#include<bits/stdc++.h>

using namespace std;

#define ll long long
#define ld long double
#define ull unsigned long long
#define for_(i, n) for(ll i = 0; i < n; ++i)
#define INF LLONG_MAX
#define NINF LLONG_MIN

const ll mod = 1e9 + 7;

//TESTED

ld f(ld x) {return x;}

ld simpson_integration(ld a, ld b, ld d){
    const ll N = 10000; // number of steps (already multiplied by 2)
    ld h = (b - a) / N;
    ld s = f(a) + f(b); // a = x_0 and b = x_2n
    for(ll i= 1; i < N; i++) { // Refer to final Simpson's formula
        double xi = a + h * i;
        s += f(xi) * ((i & 1) ? 4 : 2);
    }
    s *= h / 3.0;
    //1e6 is the decimal precision, if you want 7, the 1e7 and so on
    s*=1e6;
    s=llround(s);
    s/=1e6;
    // to print use cout << fixed << setprecision(6) << s;
    return s;
}
```

TonelliShanks

```cpp
#include<bits/stdc++.h>

using namespace std;
#define ll long long
#define INF LLONG_MAX

// TESTED


ll binpow(ll a, ll b, ll m){
    ll ans = 1;
    while(b > 0){
        if(b&1) ans = (ans*a)%m;
        a=(a*a)%m;
        b>>=1;
    }
    return ans;
}

ll TonelliShanks(ll n, ll p) {
    n %= p; if (n < 0) n += p;
    if (n == 0) return 0;
    if (binpow(n,(p-1)/2,p) != 1) return -1; // solution does not exist
    if (p % 4 == 3) return binpow(n, (p + 1) / 4, p); // trivial solution, only one factor 2
    ll c = p - 1, z = 2;
    ll s = 0;
    while(c % 2 == 0) ++s, c /= 2;
    // find a non-square mod p
    while(binpow(z,(p-1)/2, p) != p - 1) ++z;
    ll q = binpow(z, c, p); // generator subgroup, |<q>| = 2^s
    ll a = binpow(n, (c+1)/2, p); // candidate root
    ll t = binpow(n, c, p); // error from equation a^2 = n*t, if t=1, a is the root
    ll m = s;
    while(1){
        ll i = 0, temp = t;
        while(i < m && temp != 1){
            temp = (temp*temp)%p;
            i++;
        }
        if(i==0) return a;
        ll exponent = 1ll << (m-i-1);
        ll b = binpow(q, exponent, p);
        a = (a*b)%p;
        t = (((t*b)%p)*b)%p;
        q = (b*b)%p;
        m=i;
    }
}
```

Ztransform

```cpp
#include <bits/stdc++.h>
using namespace std;
#define ll long long
#define for_(i, n) for(ll i = 0; i < n; ++i)

//TESTED for gcd

//only if it is neccesary, but Z transform work on any module, and without
ll mod = 998244353;

void Z(vector<ll> &C,ll N, bool g) {
    if(g){
        for(ll d = 1; d <= N; d++) {
            for(ll m = 2*d; m <= N; m += d) C[d] = (C[d]+C[m])%mod;
        }
    }else{
        for(ll d = N; d > 0; d--) {
            for(ll m = 2*d; m <= N; m += d) C[m] = (C[d]+C[m])%mod;
        }
    }
}

void Zinv(vector<ll> &C,ll N, bool g) {
    if(g){
        for(ll d = N; d > 0; d--) {
            for(ll m = 2*d; m <= N; m += d) C[d] = (mod + (C[d]-C[m])%mod)%mod;
        }
    }else{
        for(ll d = 1; d <= N; d++) {
            for(ll m = 2*d; m <= N; m += d) C[m] = (mod + (C[m]-C[d])%mod)%mod;
        }
    }
}


vector<ll> conv(vector<ll> A, vector<ll> B, bool g) {
    ll N = A.size() - 1ll;
    Z(A, N, g);
    Z(B, N, g);
    vector<ll> C(N + 1ll);
    for (ll i = 1; i <= N; i++)  C[i] = (A[i] * B[i])%mod;
    Zinv(C, N, g);
    return C;
}
```

Bin Pow

```cpp

#include<bits/stdc++.h>

using namespace std;

#define ll long long
#define ull unsigned long long
#define INF LLONG_MAX
#define NINF LLONG_MIN

const ll mod = 1e9 + 7;

/*
Tested : Tested
*/

ll binpow(ll a, ll b, ll m){
    ll ans = 1;
    while(b > 0){
        if(b&1) ans = (ans*a)%m;
        a=(a*a)%m;
        b>>=1;
    }
    return ans;
}
```

CatalanNumbers

```cpp
#include<bits/stdc++.h>

using namespace std;

#define ll long long

//n-catalan number = (2n)!/((n+1)!n!)
ll getNCatalanNumber(ll n){
    double ans = 1;
    for(ll k = 2; k <= n; k++) ans *= (n+k)/k;
    return (ll)(ans + 0.001);
}
//TESTED
vector<ll> getCatalanNumbers(ll n){
    vector<ll> C(n+1, 0);
    C[0] = C[1] = 1;
    C[2] = 2;
    for(ll i = 3; i <= n; ++i){
        for(ll j = 0; j < i; ++j) C[i] += C[j]*C[i-j-1];
    }
    return C;
}
```

Convex Hull

```cpp
#include<bits/stdc++.h>

using namespace std;

#define ll long long


//TESTED
//Monotic Change

struct point{
    double x,  y;
};

ll orientation(const point &a, const point &b,const point &c){
    double sign = a.x*(b.y - c.y) - b.x*(a.y - c.y) + c.x*(a.y -b.y);
    if(sign > 0) return 1;
    else if( sign < 0) return -1;
    else return 0;
}

bool cw(const point &a, const point &b,const point &c,const bool &include_colider){
    ll o = orientation(a, b, c);
    return (o > 0 || include_colider && 0==0);
}

bool ccw(const point &a, const point &b,const point &c,const bool &include_colider){
    ll o = orientation(a, b, c);
    return (o < 0 || include_colider && o==0);
}

void convex_hull(vector<point> &a, bool include_colider){
    if(a.size()==1) return;
    sort(a.begin(), a.end(), [](const point &p1, const point &p2){return (p1.x==p2.x ? p1.y < p2.y : p1.x < p2.x);});
    point A = a[0], B = a[a.size()-1];
    vector<point> upper, lower;
    upper.push_back(A);
    lower.push_back(A);
    for(ll i = 1; i < a.size(); ++i){
        if(i == a.size()-1 || cw(A, a[i], B, include_colider)){
            while(upper.size() >= 2 && !(cw(upper[upper.size()-2], upper[upper.size()-1], a[i], include_colider))) upper.pop_back();
            upper.push_back(a[i]);
        }
        if(i == a.size()-1 || ccw(A, a[i], B, include_colider)){
            while(lower.size() >= 2 && !(ccw(lower[lower.size()-2], lower[lower.size()-1], a[i], include_colider))) lower.pop_back();
            lower.push_back(a[i]);
        }
    }
    if(include_colider && upper.size() == a.size()){
        reverse(a.begin(), a.end());
        return;
    }
    a.clear();
    for(ll i = 0; i < upper.size(); ++i) a.push_back(upper[i]);
    for(ll i = lower.size()-2; i >= 1; --i) a.push_back(lower[i]);
}
```

Crt

```cpp

#include<bits/stdc++.h>

using namespace std;

#define ll long long
#define ull unsigned ll
#define for_(i, n) for(ll i = 0; i < n; ++i)
#define INF LLONG_MAX
#define NINF LLONG_MIN
#define i128 __int128

const ll mod = 1e9 + 7;

//TESTED (PD: for 2x2 system but should work for any nXn)

ll gcdExtended(ll a, ll b, ll &x, ll &y){
    if(b == 0){
        x = 1;
        y = 0;
        return a;
    }
    ll x_, y_, gcd = gcdExtended(b, a%b, x_, y_);
    x = y_;
    y = x_ - y_*(a/b);
    return gcd;
}

ll mod_inv(ll a, ll m){
    ll x, y;
    ll g = gcdExtended(a, m, x, y);
    x = (x%m + m)%m;
    return x;
}

struct Congruence {
    ll b, n;
};

ll chinese_remainder_theorem(vector<Congruence> const& congruences) {
    ll N = 1;
    for (auto const& congruence : congruences) N *= congruence.n;

    ll x = 0;
    for (auto const& congruence : congruences) {
        ll b_i = congruence.b;
        ll N_i = N / congruence.n;
        ll invN_i = mod_inv(N_i, congruence.n);
        i128 aux = (b_i*N_i)%N;
        aux = (aux*invN_i)%N;
        x = (ll) (x + aux)%N;
    }
    return x;
}
```

Discrete Log

```cpp
#include<bits/stdc++.h>

using namespace std;

#define ll long long
#define ull unsigned ll
#define for_(i, n) for(ll i = 0; i < n; ++i)
#define INF LLONG_MAX
#define NINF LLONG_MIN

const ll mod = 1e9 + 7;

//Tested

ll discrete_log(ll a, ll b, ll m) {
    a %= m, b %= m;
    if (a == 0) return b == 0 ? 1 : -1;
    ll k = 1, add = 0, g;
    while ((g = gcd(a, m)) > 1) {
        if (b == k)
            return add;
        if (b % g)
            return -1;
        b /= g, m /= g, ++add;
        k = (k * 1ll * a / g) % m;
    }

    ll n = sqrt(m) + 1;
    ll an = 1;
    for_(i, n) an = (an * 1ll * a) % m;

    unordered_map<ll, ll> f2inv;
    for (ll q = 0, f2 = b; q <= n; ++q) {
        f2inv[f2] = q;
        f2 = (f2 * 1ll * a) % m;
    }

    for (ll p = 1, f1 = k; p <= n; ++p) {
        f1 = (f1 * 1ll * an) % m;
        if (f2inv.count(f1)){
            ll x = n * p - f2inv[f1] + add;
            return x;
        }
    }
    return -1;
}
```

Erathostenes-sieve

```cpp
#include<bits/stdc++.h>

using namespace std;

#define ll long long

//Tested

vector<bool> erathostenesSieve(ll n){
  vector<bool> isprime(n+1, true); vector<ll> primes;
  isprime[0] = isprime[1] = false;
  for(ll i = 2; i <= n; (i==2 ? i++ : i+=2)){
    if(isprime[i]){
      for(ll j = i*i; j <= n; j+=i) isprime[j] = false;
      primes.push_back(i);
    }
  }
  return isprime;
}
```

Factorization Pollard Rho

```cpp
#include<bits/stdc++.h>

using namespace std;

#define ll long long
#define ld long double
#define ull unsigned long long
#define for_(i, n) for(ll i = 0; i < n; ++i)
#define INF LLONG_MAX
#define NINF LLONG_MIN
#define i128 __int128
#define u128 __uint128_t

const ll mod = 1e9 + 7;

mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

//TESTED up to 1e18, run T(n) < 1e8

ll mult(ll a, ll b, ll m) {
    return (__int128)a * b % m;
}

ll f(ll x, ll c, ll m) {
    return (mult(x, x, m) + c) % m;
}

ll gcd(ll a, ll b){
    while(b) { ll r = a % b; a = b; b = r; }
    return a;
}

ll rho(ll n, ll x0=2, ll c=1) {
    ll x = x0;
    ll y = x0;
    ll g = 1;
    while (g == 1) {
        x = f(x, c, n);
        y = f(y, c, n);
        y = f(y, c, n);
        g = gcd(abs(x - y), n);
    }
    return g;
}

//algorithm for the factorization
ll brent(ll n, ll x0=2, ll c=1) {
    ll x = x0;
    ll g = 1;
    ll xs, y;

    ll m = 128;
    ll l = 1;
    ll iter = 0;
    while (g == 1 && iter<1000) {
        y = x;
        ll q = 1;
        for (ll i = 1; i < l; i++)
            x = f(x, c, n);
        ll k = 0;
        while (k < l && g == 1) {
            xs = x;
            for (ll i = 0; i < m && i < l - k; i++) {
                x = f(x, c, n);
                q = mult(q, abs(y - x), n);
            }
            g = gcd(q, n);
            k += m;
        }
        l *= 2;
        iter++;
    }
    if (g == n) {
        return n;
    }
    return g;
}

ll binpow(ll a, ll b, ll mod){
  ll ans = 1;
  while(b>0){
    if(b&1) ans = (u128)(ans)*a % mod;
    a=(u128)a*a % mod;
    b>>=1;
  }
  return ans;
}

bool checkComposite(ll p, ll a, ll s, ll q){
  ll x = binpow(a, q, p);
  if(x == 1 || x == p-1) return false;
  for(ll i = 1; i < s; ++i){
    x = (u128)x*x % p;
    if(x == p-1) return false;
  }
  return true;
}

```

Fft

```cpp
#include<bits/stdc++.h>

using namespace std;

#define ll long long
#define for_(i, n) for(ll i = 0; i < n; ++i)
#define cd complex<long double>
const double PI = 3.14159265358979323846264338327950288419716939937510;

//Tested

void fft_bottom_up(vector<cd> &c, bool inverse){
    ll N = 1;
    while(N < c.size()) N<<=1;
    c.resize(N);
    if(N<= 1) return;
    ll cnt = 0;
    for(ll j = 1; j < N; ++j){
        ll bit = N >> 1;
        while(cnt & bit){
            cnt^=bit;
            bit>>=1;
        }
        cnt^=bit;
        if(cnt < j) swap(c[cnt], c[j]);
    }

    //n is the length of the current block of the tree, thanks to bit reversal is in order, n=2 -> depth= D-1, n=4 -> depth=D-2, and so on...
    for(ll n = 2; n <= N; n<<=1){
        double ang = 2*PI/n * (inverse ? -1 : 1);
        cd wn(cos(ang), sin(ang));
        //iterate each block(node from the current depth)
        for(ll currblock = 0; currblock < N; currblock+=n){
            cd w(1);
            //iterate the node of the current block, only recover a half, because butterfly effect(como se diga la wea).
            for(ll curr = 0; curr < n/2; ++curr){
                cd u = c[currblock + curr], v = c[currblock + curr + n/2]*w;
                c[currblock + curr] = u + v;
                c[currblock + curr + n/2] = u - v;
                w*=wn;
            }
        }
    }
    if(inverse){
        for(cd &x : c) x/=N;
    }
}

vector<cd> conv(vector<cd> c1, vector<cd> c2){
    vector<cd> ans; ll N=1;
    while(N < (c1.size()+c2.size()-1ll)) N<<=1;
    c1.resize(N); c2.resize(N);
    fft_bottom_up(c1, false);
    fft_bottom_up(c2, false);
    ans.resize(N);
    for_(i, N) ans[i] = c1[i]*c2[i];
    fft_bottom_up(ans, true);
    return ans;
}

```

Gaussian Elimination

```cpp
#include<bits/stdc++.h>
#include"bin_pow.cpp"
using namespace std;

#define ll long long
#define ull unsigned long long
#define INF LLONG_MAX
#define NINF LLONG_MIN

const double EPS = 1e-18;
const int MOD = 1e9+7;

//Tested

bool gauss_jordan(vector<vector<ll>> A, vector<ll> &x) {
  ll n = A.size(), m = A[0].size() - 1;

  vector<ll> where(m, -1);
  for (ll col = 0, row = 0; col < m && row < n; ++col) {
    ll pivot = row;
    for (ll i = row; i < n; ++i){
      if (abs(A[i][col]) > abs(A[pivot][col])) pivot = i;
    }
    if(!A[pivot][col]) continue;
    for (ll i = col; i <= m; ++i) swap(A[pivot][i], A[row][i]);
    where[col] = row;

    for(ll i = 0; i < n; ++i){
      ll invm = binpow(A[row][col], MOD - 2, MOD);
      if(i != row) {
        ll c = (A[i][col] * invm) % MOD;
        for (ll j = col; j <= m; ++j) A[i][j] = ((A[i][j] - A[row][j] * c) % MOD + MOD) % MOD;
      }
    }
    ++row;
  }

  x.assign(m, 0);
  for (ll i = 0; i < m; ++i){
    if (where[i] != -1){
      ll invm = binpow(A[where[i]][i], MOD - 2, MOD);
      x[i] = (A[where[i]][m] * invm) % MOD;
    }
  }
  for(ll i=0; i<n; ++i){
    ll sum = 0;
    for(ll j=0; j<m; ++j) sum += (x[j] * A[i][j])%MOD;
    if(abs(sum - A[i][m]) > 0) return false; //No solution
  }
  return true;//Valid solution, trivial or infinity
```

Gcd

```cpp
#include<bits/stdc++.h>

using namespace std;
#define ll long long

//Tested meme
ll gcd(ll a, ll b){
    if(b==0) return a;
    return gcd(b, a%b);
}

ll gcd(ll a, ll b){
    while(b) { ll r = a % b; a = b; b = r; }
    return a;
}
```

GcdExtended

```cpp
#include<bits/stdc++.h>

using namespace std;
#define ll long long

//Tested

ll gcdExtended(ll a, ll b, ll &x, ll &y){
    if(b == 0){
        x = 1;
        y = 0;
        return a;
    }
    ll x_, y_, gcd = gcdExtended(b, a%b, x_, y_);
    x = y_;
    y = x_ - y_*(a/b);
    return gcd;
}
```

Miller Rabin

```cpp
#include<bits/stdc++.h>

using namespace std;

#define u128 __uint128_t
#define ll long long

//Tested

ll binpow(ll a, ll b, ll mod){
  ll ans = 1;
  while(b>0){
    if(b&1) ans = (u128)(ans)*a % mod;
    a=(u128)a*a % mod;
    b>>=1;
  }
  return ans;
}

bool checkComposite(ll p, ll a, ll s, ll q){
  ll x = binpow(a, q, p);
  if(x == 1 || x == p-1) return false;
  for(ll i = 1; i < s; ++i){
    x = (u128)x*x % p;
    if(x == p-1) return false;
  }
  return true;
}

bool millerRabin(ll p, ll maxIter=5){
  if(p<4) return p==2 || p==3;
  ll s=0,q = p-1;
  while(!(q&1)){
    s++;
    q>>=1;
  }
  for(ll i = 0; i < maxIter; ++i){
    ll a = 2 + rand() % (p-3);
    if(checkComposite(p, a, s, q)) return false;
  }
  return true;
}

bool deterministicMillerRabin(ll p){
  if(p<4) return p==2 || p==3;
  ll s=0,q = p-1;
  while(!(q&1)){
    s++;
    q>>=1;
  }
  for(ll a : {2,3,5,7,11,13,17,19,23,29,31,37}){
    if(p==a) return true;
    if(checkComposite(p, a, s, q)) return false;
  }
  return true;
}
```

Mobius

```cpp

#include<bits/stdc++.h>

using namespace std;

#define ll long long
#define ull unsigned long long
#define for_(i, n) for(ll i = 0; i < n; ++i)
#define INF LLONG_MAX
#define NINF LLONG_MIN

const ll mod = 1e9 + 7;

//Tested

vector<ll> mobius(ll n){
    vector<ll> mob(n+1, 1);
    mob[1] = 1;
    vector<bool> isprime(n+1, true);
    isprime[0]=isprime[1] = false;
    for(ll i = 2; i < n+1; i++){
        if(isprime[i]){
            for(ll j = i; j < n+1; j+=i){
                mob[j] *= -1;
                isprime[j] = false;
            }
            for(ll j = i*i; j < n+1; j += i*i) mob[j] = 0;
        }
    }
    return mob;
}
```

Ntt

```cpp
#include<bits/stdc++.h>

using namespace std;

#define ll long long
#define for_(i, n) for(ll i = 0; i < n; ++i)

//TESTED

const ll mod = 998244353, generator = 3, rootn = 3; // for N max = 2^23
/*more modelus primes of the form p = c*2^m + 1 (generators calculated with algorithm in math section):
const ll mod = 7340033, generator = 3, rootn = 3; // for N max = 2^20
const ll mod = 167772161, generator = 3, rootn = 3; // for N max = 2^25
const ll mod = 469762049, generator = 3, rootn = 3; // for N max = 2^26
*/

ll binpow(ll a, ll e){
    ll ans = 1;
    while(e > 0){
        if(e&1) ans = (ans * a)%mod;
        a = (a*a)%mod;
        e>>=1;
    }
    return ans;
}

ll inv(ll a){return binpow(a, mod-2);}


void ntt(vector<ll> &c, bool invert){
    ll N = 1;
    while(N < c.size())N<<=1;
    c.resize(N);
    ll cnt = 0;
    for(ll j = 1; j < N; ++j){
        ll bit = N>>1;
        while(bit&cnt){
            cnt^=bit;
            bit>>=1;
        }
        cnt^=bit;
        if(cnt < j) swap(c[cnt], c[j]);
    }

    for(ll n = 2; n <= N; n<<=1){
        ll wn = binpow(rootn, (mod-1)/n);
        if(invert) wn = inv(wn);
        for(ll currblock = 0; currblock < N; currblock+=n){
            ll w = 1;
            for(ll i = 0; i < n/2; ++i){
                ll u = c[currblock + i]%mod, v = (w*c[currblock + i + n/2])%mod;
                c[currblock + i] = (u + v)%mod;
                c[currblock + i + n/2] = ((u - v)%mod + mod)%mod;
                w = (w*wn)%mod;
            }
        }
    }
    if(invert){
        for(ll &x : c) x = (x*inv(N))%mod;
    }
}

vector<ll> conv(vector<ll> c1, vector<ll> c2){
    ll N=1;
    while(N <= (c1.size() + c2.size()-1ll)) N<<=1;
    c1.resize(N); c2.resize(N);
    ntt(c1, false);
    ntt(c2, false);
    vector<ll> ans(N, 0);
    for_(i, N) ans[i] = (c1[i] * c2[i])%mod;
    ntt(ans, true);
    return ans;
}
```

Primitive Root

```cpp
#include<bits/stdc++.h>
#include<iostream>

using namespace std;

#define ll long long
#define ull unsigned ll
#define for_(i, n) for(ll i = 0; i < n; ++i)
#define INF LLONG_MAX
#define NINF LLONG_MIN

const ll mod = 1e9 + 7;

//TESTED

ll binpow(ll a, ll b, ll m){
    ll ans = 1;
    while(b > 0){
        if(b&1) ans = (ans*a)%m;
        a=(a*a)%m;
        b>>=1;
    }
    return ans;
}

vector<ll> tot_euler(ll n){
    vector<ll> tot(n+1, 0);
    for_(i, n+1) tot[i]=i;
    for(ll i = 2; i <= n; ++i){
        if(tot[i] == i){
            for(ll j = i; j <= n; j+=i) tot[j] -= tot[j]/i;
        }
    }
    return tot;
}

ll generator_(ll n) {
    vector<int> facts;
    ll phi_n = tot_euler(n)[n], m = phi_n;
    for(ll i=2; i*i<=m; ++i){
        if(m % i == 0) {
            facts.push_back(i);
            while (m % i == 0) m/= i;
        }
    }
    if(m > 1) facts.push_back(m);
    for (ll g=2; g<=n; ++g) {
        bool ok = true;
        for (size_t i=0; i<facts.size() && ok; ++i) ok &= binpow(g, phi_n/facts[i], n) != 1;
        if (ok)  return g;
    }
    return -1;
}
```

Template

```cpp
#include<bits/stdc++.h>

using namespace std;

#define ll long long
#define ld long double
#define for_(i, n) for(ll i = 0; i < n; ++i)
#define ull unsigned long long
#define INF LLONG_MAX
#define NINF LLONG_MIN
#define i128 __int128
#define u128 __uint128_t
#define cd complex<ld>
const double PI = 3.14159265358979323846264338327950288419716939937510;
const double EPS = 1e-18;
//mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

```

Tot Euler

```cpp
#include<bits/stdc++.h>

using namespace std;

#define ll long long
#define ull unsigned long long
#define for_(i, n) for(ll i = 0; i < n; ++i)
#define INF LLONG_MAX
#define NINF LLONG_MIN

const ll mod = 1e9 + 7;

//Tested

vector<ll> tot_euler(ll n){
    vector<ll> tot(n+1, 0);
    for_(i, n+1) tot[i]=i;
    for(ll i = 2; i <= n; ++i){
        if(tot[i] == i){
            for(ll j = i; j <= n; j+=i) tot[j] -= tot[j]/i;
        }
    }
    return tot;
}
```

Strings

Aho-corasick

```cpp
#define sz(x) ((int)(x).size())
#define rep(i, a, b) for (int i = (a); i < (b); ++i)
typedef vector<int> vi;
struct AhoCorasick {
  enum { alpha = 26, first = 'a' };  // change this!
  struct Node {
    // (nmatches is optional)
    int back, next[alpha], start = -1, end = -1, nmatches = 0;
    Node(int v) { memset(next, v, sizeof(next)); }
  };
  vector<Node> N;
  vi backp;
  void insert(string& s, int j) {
    assert(!s.empty());
    int n = 0;
    for (char c : s) {
      int& m = N[n].next[c - first];
      if (m == -1) {
        n = m = sz(N);
        N.emplace_back(-1);
      } else
        n = m;
    }
    if (N[n].end == -1) N[n].start = j;
    backp.push_back(N[n].end);
    N[n].end = j;
    N[n].nmatches++;
  }
  AhoCorasick(vector<string>& pat) : N(1, -1) {
    rep(i, 0, sz(pat)) insert(pat[i], i);
    N[0].back = sz(N);
    N.emplace_back(0);

    queue<int> q;
    for (q.push(0); !q.empty(); q.pop()) {
      int n = q.front(), prev = N[n].back;
      rep(i, 0, alpha) {
        int &ed = N[n].next[i], y = N[prev].next[i];
        if (ed == -1)
          ed = y;
        else {
          N[ed].back = y;
          if (N[ed].end == -1) {
            N[ed].end = N[y].end;
          } else {
            backp[N[ed].start] = N[y].end;
          }
          N[ed].nmatches += N[y].nmatches;
          q.push(ed);
        }
      }
    }
  }
  vi find(string word) {
    int n = 0;
    vi res;  // ll count = 0;
    for (char c : word) {
      n = N[n].next[c - first];
      res.push_back(N[n].end);
      // count += N[n].nmatches;
    }
    return res;
  }
  vector<vi> findAll(vector<string>& pat, string word) {
    vi r = find(word);
    vector<vi> res(sz(word));
    rep(i, 0, sz(word)) {
      int ind = r[i];
      while (ind != -1) {
        res[i - sz(pat[ind]) + 1].push_back(ind);
        ind = backp[ind];
      }
    }
    return res;
  }
};
```

Kmp

```cpp
struct KMP {
  vector<int> lps;
  string s;
  KMP(string s) : s(s) {
    int n = s.size();
    lps.assign(n, 0);
    int len = 0, i = 1;
    while (i < n) {
      if (s[i] == s[len]) {
        lps[i++] = ++len;
      } else {
        if (len != 0)
          len = lps[len - 1];
        else
          lps[i++] = 0;
      }
    }
  }
};
```

RollingHashing

```cpp
template <class T>
struct rolling_hashing {
  int base, mod;
  vector<long long> p, H;
  int n;
  rolling_hashing(const T& s, int b, int m) : base(b), mod(m), n(s.size()) {
    p.assign(n + 1, 1);
    H.assign(n + 1, 0);
    for (int i = 0; i < n; ++i) {
      H[i + 1] = (H[i] * base + s[i]) % mod;
      p[i + 1] = (p[i] * base) % mod;
    }
  }
  int get(int l, int r) {
    int res = (H[r + 1] - H[l] * p[r - l + 1]) % mod;
    if (res < 0) res += mod;
    return res;
  }
};
```

Trie

```cpp
struct trie {
  vector<vector<ll>> tree;
  vector<ll> counts;
  trie() {
    tree.push_back(vector<ll>(26, -1));
    counts.push_back(0);
  };
  void insert(string& s) {
    ll i = 0, u = 0;
    while (i < s.size()) {
      char c = s[i];
      if (tree[u][c - 'a'] == -1) {
        ll pos = tree.size();
        tree.push_back(vector<ll>(26, -1));
        counts.push_back(0);
        tree[u][c - 'a'] = pos;
      }
      i++;
      u = tree[u][c - 'a'];
    }
    counts[u]++;  // count final character
  }
  ll search(string& s) {
    ll i = 0, u = 0;
    ll count_ends = 0;
    while (i < s.size()) {
      char c = s[i];
      if (tree[u][c - 'a'] != -1) {
        // If you need to know how many trailing characters it goes through,
        // uncomment. count_ends += counts[u];
        u = tree[u][c - 'a'];
        i++;
      } else
        break;
    }
    if (i == s.size()) count_ends += counts[u];
    return count_ends;
  }
};
```

Zfunction

```cpp
struct Z {
  string s;
  vector<int> z;
  ll n;
  Z(string s) : s(s) {
    n = s.size();
    z.assign(n, 0);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
      if (i < r) {
        z[i] = min(r - i, z[i - l]);
      }
      while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
        z[i]++;
      }
      if (i + z[i] > r) {
        l = i;
        r = i + z[i];
      }
    }
  }
};
```

Programacion Dinamica

Knappsack

```cpp
void solve() {
  ll n, w;
  cin >> n >> w;
  vector<pair<ll, ll>> v(n);
  for (auto& [a, b] : v) cin >> a >> b;
  vector<ll> ans(w + 2, 0);
  ll ans_ = 0;
  for (auto [vi, wi] : v) {
    for (ll j = w; j >= wi; j--) {
      ans[j] = max(ans[j], ans[j - wi] + vi);
      ans_ = max(ans_, ans[j]);
    }
  }
  cout << ans_ << endl;
}
```

Lis

```cpp
template <class I>
vector<int> LIS(const vector<I>& S) {
  if (S.empty()) return {};
  vector<int> prev(S.size());
  vector<pair<I, int>> res;
  for (int i = 0; i < S.size(); i++) {
    auto it = lower_bound(res.begin(), res.end(), pair<I, int>{S[i], i});
    if (it == res.end()) res.emplace_back(), it = res.end() - 1;
    *it = {S[i], i};
    prev[i] = (it == res.begin() ? 0 : (it - 1)->second);
  }
  int L = res.size(), cur = res.back().second;
  vector<int> ans(L);
  while (L--) ans[L] = cur, cur = prev[cur];
  /* Recover the sequence
  for (int i = 0; i+1 < ans.size(); i++)
    ans[i] = S[ans[i]];
  */
  return ans;
}
```

Lis Length K

```cpp
vector<fenwick_tree> bit(k + 1, fenwick_tree(m));

ll answer = 0;
for (ll x : v) {
  ll value = compress(x);  // or already compressed
  vector<ll> current(k + 1);
  current[1] = 1;
  for (ll len = 2; len <= k; len++) {
    current[len] = bit[len - 1].sum(value - 1);
  }
  for (ll len = 1; len <= k; len++) {
    bit[len].add(value, current[len]);
  }
  answer += current[k];
}
cout << answer << endl;
```

Miscelaneo

Busqueda Binaria

```cpp
int bs(vector<ll>& v, int b) {
  int l = 0, r = v.size();
  while (l < r) {
    int mid = (l + r) / 2;
    if (v[mid] >= b)
      r = mid;
    else
      l = mid + 1;
  }
  return l;
}
```

Random

```cpp
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
mt19937_64 rng64(chrono::steady_clock::now().time_since_epoch().count());

// Número aleatorio entero [a, b]
int random(int a, int b) {
    return uniform_int_distribution<int>(a, b)(rng);
}
// Número aleatorio long long [a, b]
ll random(ll a, ll b) {
    return uniform_int_distribution<ll>(a, b)(rng64);
}
// shuffle a un vector v
shuffle(v.begin(), v.end(), rng);

```

Time

```cpp
using Clock = chrono::steady_clock;
using TimePoint = Clock::time_point;
// Obtener instante actual
TimePoint getTime() { return Clock::now(); }

// Diferencia en segundos
double elapsedSeconds(TimePoint start) {
  return chrono::duration<double>(Clock::now() - start).count();
}

// Diferencia en milisegundos
ll elapsedMilliseconds(TimePoint start) {
  return chrono::duration_cast<chrono::milliseconds>(Clock::now() - start)
      .count();
}

// Diferencia en microsegundos
ll elapsedMicroseconds(TimePoint start) {
  return chrono::duration_cast<chrono::microseconds>(Clock::now() - start)
      .count();
}

// Diferencia en nanosegundos
ll elapsedNanoseconds(TimePoint start) {
  return chrono::duration_cast<chrono::nanoseconds>(Clock::now() - start)
      s.count();
}

// Dormir durante cierta cantidad de tiempo
void sleepSeconds(int s) { this_thread::sleep_for(chrono::seconds(s)); }

void sleepMilliseconds(int ms) {
  this_thread::sleep_for(chrono::milliseconds(ms));
}
```